# Apache Flink - Gemini Notebook

Exported on: 8/3/2026, 5:21:30 AM

> Explain how fault tolerant flink cluster work and how they can be setup in one or more unix servers. How many nodes does the cluster need to be reliably fault tolerant?

Fault tolerance in Apache Flink is achieved through a combination of **consistent checkpointing** and a **high-availability (HA)** cluster architecture [1] [2]. This ensures that even if individual processes or entire machines fail, the application can restore its internal state and continue processing with exactly-once consistency guarantees [3] [4].

How Fault-Tolerant Flink Clusters Work

- **Checkpointing Mechanism:** Flink uses the **Chandy-Lamport algorithm** to create distributed snapshots of the entire application state without stopping execution [5]. **Checkpoint barriers** are injected into the data stream at the sources and flow through the operator graph; when an operator receives these barriers from all its inputs, it snapshots its local state to a remote, persistent storage [6].
- **State Recovery:** In the event of a task failure, Flink restarts the application, resets the state of all tasks to the latest successful checkpoint, and replays input data from the point the checkpoint was taken [7]. This requires **resettable data sources** (like Apache Kafka) to provide data from previous offsets [3].
- **JobManager High Availability:** Because the JobManager is a single point of failure that coordinates execution and manages metadata, a reliably fault-tolerant cluster must run in **HA mode** [8] [9]. This mode uses **Apache ZooKeeper** for leader election (selecting an active JobManager from several standby instances) and for storing pointers to the metadata in remote storage [9] [10].

Setting Up a Cluster on Unix Servers

To set up a fault-tolerant standalone cluster on one or more Unix servers, follow these steps:

1. **Preparation:** Ensure Java 8 (or higher) is installed and that passwordless SSH is configured between the master and all worker nodes [11].
2. **Configuration:**
   - In `./conf/flink-conf.yaml`, set `jobmanager.rpc.address` to the master node's IP [12].
   - Enable HA by setting `high-availability: zookeeper` and providing the `high-availability.zookeeper.quorum` (a list of ZooKeeper nodes) [13] [14].
   - Define a remote storage path for metadata and checkpoints using `high-availability.storageDir` (e.g., an HDFS or S3 path) [13] [14].
   - List the hostnames of all worker servers in the `./conf/slaves` file [15].
3. **Deployment:**

- Start the ZooKeeper quorum (using `./bin/start-zookeeper-quorum.sh` for testing or a dedicated external cluster for production) [10] [13].
- Run `./bin/start-cluster.sh` from the master node; this script will start the master process locally and the TaskManager processes on the remote slaves via SSH [16] [12].

Required Number of Nodes for Reliability

For a cluster to be **reliably fault-tolerant**, it must be able to withstand the loss of either a JobManager or a TaskManager without total system failure:

- **JobManager Reliability:** You should run at least **two or three nodes** acting as masters [17] [18]. While only one JobManager is active at a time, standby instances must be ready to take over via ZooKeeper election [18]. A standard ZooKeeper quorum typically requires at least **three nodes** to provide reliable consensus [10] [13].
- **TaskManager Reliability:** The number of worker nodes depends on your job's parallelism. For reliability, you need **standby processing slots** distributed across multiple nodes [14] [19]. If a node fails, Flink must find enough idle slots on the remaining nodes to restart the lost tasks; if no slots are available, the job cannot resume until a new TaskManager registers [14] [19].
- **Minimum Practical Count:** While a single-node "cluster" is possible for development, a reliably fault-tolerant production environment typically requires a minimum of **three to five nodes** (to host a ZooKeeper quorum, redundant JobManagers, and enough worker slots to handle task redistribution after a node loss) [10].

When using apache flink with my java backend application, will adding flink as a gradle dependency auto generate the java classes from xml or some other schme or does that have to be created manually by the java backend app's developer?

Adding Apache Flink as a dependency to your project does **not automatically generate Java classes** from XML or other schemas [20] [21]. Instead, the developer is generally responsible for creating the data classes used in the application manually, or using external tools for specific formats like Apache Avro [22] [23].

Here is how data classes and schemas are typically handled when developing with Flink:

1. Manual Creation (POJOs and Tuples)

For most applications, developers manually create **POJOs (Plain Old Java Objects)** or use built-in **Tuple** classes to represent their data [21]. Flink recognizes a class as a POJO if it meets specific criteria, such as being public, having a public no-argument constructor, and having public fields or standard getters and setters [23].

2. Schema-Based Generation (Apache Avro)

While Flink itself does not generate classes from schemas, it has **built-in support for classes generated by Apache Avro** [21] [23]. If you have a schema defined in Avro, you would typically use an external Maven or Gradle plugin (not the Flink library itself) to generate the Java classes from that schema [23]. Flink can then automatically identify these generated classes and handle them efficiently as POJOs [23] [24].

3. Flink Archetypes and Skeleton Files

When you bootstrap a new Flink project using Maven (or Gradle equivalents), the process generates **skeleton files** (like `BatchJob.java` or `StreamingJob.java`) to serve as a starting point for your application logic [25] [26]. However, these are templates for the Flink "job" itself, not for your specific data models or schemas [26].

4. Serialization and Type Information

Rather than generating classes, Flink's dependency includes a **Type Extraction System** [20] [27]. This system analyzes the Java classes you have provided (manual or generated) to automatically create serializers and deserializers, which allows Flink to move your data objects over the network or save them to checkpoints without you having to write that logic yourself [20] [28].

In summary, you will need to **manually define your data model** or use a schema-specific generator (like Avro) alongside Flink, as the Flink dependency focuses on the runtime processing and API rather than schema-to-code generation [22] [23] [27].

What do you mean it takes any public class with public fields and getter setters to be state models. Thats too broad. Isnt it controlled by some annotation or at least limited to some package?

The sources confirm that Flink's criteria for identifying a class as a **POJO (Plain Old Java Object)** for its state models and data streams are indeed primarily structural and broad, rather than being controlled by a specific Flink-internal annotation or package restriction [29].

Criteria for POJO Recognition

Flink automatically identifies a class as a POJO if it meets these four specific conditions:

1. **Public Class:** The class must be declared as `public` [29].
2. **Public No-Arg Constructor:** It must have a public constructor with no arguments (a default constructor) [29].
3. **Field Accessibility:** All fields must either be `public` or accessible through standard getters and setters that follow the naming scheme `getY()` and `setY(X x)` for a field `y` [29].
4. **Supported Field Types:** All fields within the class must be of types that Flink also supports [29].

Why the Criteria Are So Broad

While this might seem overly inclusive, Flink uses these structural rules to power its **Type Extraction System** [30]. By identifying POJOs this way, Flink can automatically generate specialized, high-performance serializers and deserializers for your data [31]. This allows Flink to handle these objects more efficiently than if it had to fall back on general-purpose serialization like **Kryo**, which is used for types it cannot identify [32].

Control and Overrides

If you find this automatic detection too broad or if the type extractor fails (e.g., due to Java's type erasure in generic types), Flink provides ways to manually control type information:

- **Explicit Type Information:** You can use the `.returns()` method when defining a transformation or implement the `ResultTypeQueryable` interface to explicitly tell Flink how to handle a specific class [33].
- **Connector-Specific Annotations:** While the core Flink engine doesn't require annotations, specific connectors might. For example, if you use the **Apache Cassandra Sink**, your POJOs *must* use Cassandra-specific annotations like `@Table` and `@Column` so that the connector's object mapper can correctly link your Java fields to database columns [34].

In summary, there is no single `@StateModel` or `@Pojo` annotation required for general use; Flink relies on the **Java Bean-like structure** of your classes to understand how to serialize your state [29].

From your statement, "All fields within the class must be of types that Flink also supports.".
What are the types Flink supports?

Apache Flink's type system is designed to handle a wide range of data types efficiently by generating specialized serializers and deserializers. The sources identify the following categories of supported data types:

1. Primitives

Flink supports all common Java and Scala **primitive types**, including:

- **Integers:** `Int`, `Integer`, `Long` [35]
- **Floating-point:** `Double`, `Float` [35] [36]
- **Others:** `String`, `Boolean`, `Byte`, `Short`, and `Char` [35]

2. Java and Scala Tuples

Tuples are composite types with a fixed number of fields:

- **Java Tuples:** Flink provides its own implementation for Java, ranging from `Tuple1` to `Tuple25`. These are mutable, allowing for field reuse [37].
- **Scala Tuples:** Flink natively supports standard Scala tuples [35].

3. Scala Case Classes

Flink supports **Scala case classes**, which are handled similarly to tuples but allow fields to be accessed by name rather than position [38].

4. POJOs (Plain Old Java Objects)

Flink identifies a class as a POJO if it is public, has a public no-arg constructor, and has public fields or standard getters/setters [39]. This category also includes **classes generated by Apache Avro**, which Flink identifies and handles as POJOs automatically [39].

5. Special Types

Several other types are specially supported to ensure efficient handling:

- **Collections:** `ArrayList`, `HashMap`, and `Enum` types [40].
- **Arrays:** Primitive and object array types [40].
- **Hadoop Types:** Types that implement the `Hadoop Writable` interface [40].
- **Scala-specifics:** Support for `Either`, `Option`, and `Try` [40].

Fallback: Kryo Serialization

If a data type does not fall into any of the above categories, Flink treats it as a **generic type** and uses the **Kryo serialization framework** as a fallback [41]. However, the sources note that Kryo is generally less efficient than Flink's specialized serializers and should be avoided if possible [41].

Is there a central place where to defines the list of states/objects i need flink to store/track?

No, there is **no single central registry** or configuration file in Apache Flink where you define a master list of all states or objects for the entire cluster [42]. Instead, Flink's state management is decentralized and **associated with specific operators** within your application logic [42].

Here is how and where state is defined and tracked:

1. Definition at the Operator Level

State is local to the function or operator that uses it [43] [44]. If you want Flink to track a piece of data (like a counter or a user profile), you must **register that state** within the specific operator's implementation [42]. This is typically done in the `open()` method of a `RichFunction` using a `StateDescriptor` [45] [46].

2. Registration via RuntimeContext

For **keyed state** (state tied to a specific key like a user ID), you register it via the `RuntimeContext` [45] [47]. You provide a `StateDescriptor` which includes:

- **A unique name** for the state (scoped to that specific operator) [45].
- **The type of state** (e.g., `ValueState`, `ListState`, or `MapState`) [48] [49].
- **Type Information** so Flink knows how to serialize the objects [45].

3. Registration via Interfaces

For **operator state** (state tied to a parallel task rather than a key), you typically implement specific interfaces:

- **CheckpointedFunction:** This is the most flexible and lowest-level interface for registering both keyed and operator state [50].
- **ListCheckpointed:** A simpler interface for functions that only need operator list state [51].

4. Automatic Tracking (Type Extraction)

While you don't list your objects in a central place, Flink's **Type Extraction System** automatically "tracks" the objects you use [52]. It analyzes your function signatures (the input and output types) to generate `TypeInformation`, which tells Flink how to efficiently move those objects over the network or save them to checkpoints [52] [53].

Summary of Where State "Lives"

- **In Code:** Definitions are spread across your individual `Map`, `FlatMap`, or `Process` functions [47] [54] [55].
- **At Runtime:** The **State Backend** (like RocksDB or the Heap) is the "place" that actually stores and manages these objects across the cluster while the job is running [56] [57].
- **For Recovery:Checkpoints and Savepoints** create a distributed snapshot of all these local states and store them in a persistent remote filesystem (like HDFS or S3) [57] [58].

Show me using an example of class Order. The app receives orders on a kafka topic and cosumes it, validates it, enriches it, and send it our for fullfillment. It updates the status field in the order object at each step. This orde object must remain accessible to the app and must be persisted to a traditional rdbms as a write-behind strategy that is eventually consistent with the order data in flink. Eventual but still really quick.

To implement this order processing workflow in Apache Flink, you should use a **stateful event-driven architecture**. This involves consuming the stream, managing the `Order` object in **Keyed State**, and using a sink to persist updates to your RDBMS [59] [60] [61].

1. The Order Data Model (POJO)

For Flink to handle your `Order` class efficiently, it must be a **POJO**. This means it must be `public`, have a **public no-argument constructor**, and all fields must be `public` or accessible via standard getters and setters [62] [63].

```
public class Order {
    public String orderId;
    public String status;
    public String details;
    // Standard getters, setters, and no-arg constructor [5]
}
```

2. Consuming and Processing the Stream

The application starts by ingesting the stream from a **Kafka Source** [64] [65]. You then use `keyBy(orderId)` to ensure all events for a specific order are handled by the same parallel task, which allows the app to maintain the order's state [66] [67].

The core logic is implemented in a **KeyedProcessFunction**, which provides access to **ValueState** to keep the `Order` object accessible throughout its lifecycle [68] [69].

```
public class OrderProcessor extends KeyedProcessFunction<String, Order, Order> {
    private transient ValueState<Order> orderState;

    @Override
    public void open(Configuration conf) {
        // Register state handle [12]
        ValueStateDescriptor<Order> desc = new ValueStateDescriptor<>
("orderState", Order.class);
        orderState = getRuntimeContext().getState(desc);
    }

    @Override
    public void processElement(Order newOrder, Context ctx, Collector<Order> out)
throws Exception {
        Order current = orderState.value();
        if (current == null) { current = newOrder; }
```

```
        // Step 1: Validate [13]
        current.status = "VALIDATED";
        orderState.update(current);
        out.collect(current); // Emit for write-behind sync

        // Step 2: Enrich (could involve Async I/O for speed) [14, 15]
        current.details += " - Enriched Data";
        current.status = "ENRICHED";
        orderState.update(current);
        out.collect(current);

        // Step 3: Fulfillment [13]
        current.status = "SENT_FOR_FULFILLMENT";
        orderState.update(current);
        out.collect(current);
    }
}
```

3. Write-Behind RDBMS Strategy

To achieve a "really quick" write-behind strategy that is eventually consistent, you pipe the output of your `OrderProcessor` into a **JDBC Sink** [70] [71] [72].

- **Mechanism:** Every time `out.collect(current)` is called in the code above, the updated `Order` object is sent to the sink.
- **Idempotent Updates:** The RDBMS sink should use **UPSERT** (or `INSERT ... ON DUPLICATE KEY UPDATE`) semantics [72] [73] [74]. This ensures that even if Flink replays a status update during recovery, the database simply overwrites the record with the same state, maintaining consistency [75] [76].
- **Performance:** By emitting the object immediately after each state change, the RDBMS is updated in near real-time [77] [78].

4. Reliability and Consistency

To ensure this process is reliably fault-tolerant:

- **Checkpointing:** Enable exactly-once checkpointing [79] [80]. This persists the `ValueState<Order>` to remote storage (like HDFS or S3) [81] [82].
- **Resettable Sources:** Using Kafka as the source allows Flink to replay data from the last successful checkpoint if a crash occurs, ensuring no orders are lost [83] [84] [85].
- **State Cleanup:** To prevent the RDBMS and Flink state from growing indefinitely, you should register a **timer** to clear the `orderState` after the order reaches a terminal status (e.g., "DELIVERED") [86] [87] [88].

1. What does out.collect(...) do?
2. Why is ValueState transient?
3. Is the ValueState only accessible from within the KeyedProcessFunction?

Based on the provided sources and our conversation history, here are the answers to your questions:

1. What does `out.collect(...)` do?

The `out.collect(...)` method is used to **emit result records** from an operator to the next stage in the dataflow graph, such as a downstream transformation or a data sink [89] [90]. In the context of a `ProcessFunction` or `FlatMapFunction`, this method allows the function to produce zero, one, or multiple output events for every single input event it receives [89] [90].

2. Why is `ValueState<Order>` transient?

In Java, the `transient` keyword indicates that a field should not be serialized when the object containing it is serialized. In Flink, function objects (like your `OrderProcessor`) are **serialized using Java serialization** on the client side and then shipped over the network to the worker nodes (TaskManagers) [91] [92].

The `ValueState` handle itself is a **runtime object** provided by Flink's `RuntimeContext` once the task is actually running on a worker node [93] [94]. Because this handle is specific to the runtime environment of the TaskManager and does not exist when the code is first serialized on your machine, it must be marked `transient` to prevent the Java serializer from trying (and failing) to include it in the initial serialized package [92] [94]. Instead, the handle is initialized "fresh" on the worker node inside the `open()` method [94].

3. Is the `ValueState` only accessible from within the `KeyedProcessFunction`?

Technically, **keyed state** (like `ValueState`) is accessible from any **rich function** that is applied to a `KeyedStream`, not just `KeyedProcessFunction` [95] [96]. However, its accessibility is strictly scoped:

- **Operator Scoping:** Keyed state is **scoped to the specific operator** that registered it [93] [97]. One operator cannot directly access the internal state of another operator in the Flink job [98].
- **Key Scoping:** Access is automatically restricted to the **context of the current key** being processed [99]. When your code calls `orderState.value()`, Flink internally ensures you only see the data belonging to the specific `orderId` currently being handled by that task [99] [100].
- **External Access:** The one major exception is **Queryable State** [101]. If you explicitly configure your state to be queryable, external applications can perform read-only lookups on that state while the Flink job is running [101] [102].

1. Define operator clearly.
2. What is a good use case for Queryable State in my case of Order processing?
3. How is the flow stating from kafka to KeyedProcessFunction to jdbc sink defined and where is it defined? I have used Apache camel and in that we used to define the entire route each incoming upstream message would follow in a RouteBuilder class. Anything like that?

Based on the provided source and our conversation history, here are the answers to your questions:

1. Definition of an Operator

In Flink, **operators are the basic functional units** of a dataflow application[103]. They are the nodes in a directed graph (a dataflow program) where each node represents a specific computation and the edges represent the data dependencies between them[103].

- **Function:** Operators consume data from their inputs, perform a computation or transformation on that data, and then produce data to their outputs for further processing[103].
- **Special Types:** Operators that do not have input ports are called **data sources**, and those without output ports are called **data sinks**[103].
- **State Scoping:** As discussed in our previous conversation, managed state is always associated with a specific operator; one operator cannot directly access the internal state of another[104][105].

2. Use Case for Queryable State in Order Processing

In your order processing scenario, a great use case for **Queryable State** would be a **real-time order status dashboard**[106].

- **The Scenario:** Since you are already storing the `Order` object (including its status field) in a `ValueState<Order>` within your `KeyedProcessFunction`, this data is "live" inside Flink[107].
- **The Benefit:** Instead of having an external application query your RDBMS (which is being updated via a write-behind strategy), that application could use a `QueryableStateClient` to perform a **read-only point lookup** directly against Flink's internal state using the `orderId`[106][108][109].
- **Efficiency:** This allows you to serve the absolute latest status of an order to a customer or support agent with millisecond latency, without needing to maintain a separate, high-speed external key-value store just for status lookups[106][108].

3. Defining the Flow: Kafka to Sink

Unlike Apache Camel, which uses a `RouteBuilder` class to define routes, Flink applications are typically defined within a **standard Java or Scala main() method**[110][111].

- **Assembling the Dataflow:** You define the "route" (the dataflow graph) by **chaining transformations** on a `DataStream` object[112][113][114]. For your specific case, the definition would look like this in your code:
  1. **Source:** `env.addSource(new FlinkKafkaConsumer(...))` defines the start[115][116].
  2. **Transformation:** Chaining `.keyBy(orderId)` and `.process(new OrderProcessor())` defines the logic and state management[107][113].

3. **Sink:** Ending the chain with `.addSink(new JdbcSink(...))` defines the destination[115][117].

- **Lazy Execution:** These API calls do not trigger immediate processing[118]. Instead, they lazily **construct an execution plan** (the `JobGraph`)[118]. The entire "route" is only submitted to the cluster for execution when you explicitly call **env.execute()** at the very end of your `main()` method[111][118].
- **Structure:** While not a required class like `RouteBuilder`, it is best practice to structure the application logic by: (1) Setting up the execution environment, (2) Reading sources, (3) Applying transformations, (4) Defining sinks, and (5) Calling execute[111].

Pertinent to this point,

```
"Structure: While not a required class like RouteBuilder, it is best practice to structure the application logic by: (1) Setting up the execution environment, (2) Reading sources, (3) Applying transformations, (4) Defining sinks, and (5) Calling execute"
```

Show me as sample code in main() for my Order processing use case as to how you would setup the full route in a standard, readable and non-boilerplate way?

Following the best practices for structuring a Flink application, your **Order processing** route would be assembled within a standard Java `main()` method[119][120]. This fluent API approach allows you to define a complex distributed dataflow in a readable, non-boilerplate way.

Here is how you would set up the full route using the five-step structure:

Order Processing Application `main()`

```
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import org.apache.flink.connector.jdbc.JdbcSink;
import org.apache.flink.connector.jdbc.JdbcConnectionOptions;
import org.apache.flink.connector.jdbc.JdbcExecutionOptions;
import org.apache.flink.streaming.connectors.kafka.FlinkKafkaConsumer;
import java.util.Properties;

public class OrderProcessingJob {
    public static void main(String[] args) throws Exception {

        // 1. Set up the execution environment [2, 3]
        // This automatically detects if running locally or on a cluster
        final StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();

        // Ensure fault tolerance by enabling checkpointing [4]
        env.enableCheckpointing(5000);

        // 2. Read sources (Kafka) [5, 6]
        Properties props = new Properties();
        props.setProperty("bootstrap.servers", "localhost:9092");
        props.setProperty("group.id", "order-processor-group");

        DataStream<Order> orderStream = env.addSource(new FlinkKafkaConsumer<>(
                "incoming-orders",
                new OrderDeserializationSchema(), // Custom POJO deserializer
                props
        ));
```

```
        // 3. Apply transformations [2, 7]
        // Chaining: Key by orderId -> Stateful validation, enrichment,
fulfillment
        DataStream<Order> processedOrders = orderStream
                .keyBy(order -> order.orderId) // Logical partitioning [8]
                .process(new OrderProcessor()); // KeyedProcessFunction with
ValueState [9]

        // 4. Define sinks (Write-Behind JDBC Sink) [2, 10, 11]
        // Emits updates to RDBMS using an UPSERT (Idempotent) strategy
        processedOrders.addSink(JdbcSink.sink(
                "INSERT INTO orders (id, status, details) VALUES (?, ?, ?) " +
                "ON DUPLICATE KEY UPDATE status = VALUES(status), details =
VALUES(details)",
                (statement, order) -> {
                    statement.setString(1, order.orderId);
                    statement.setString(2, order.status);
                    statement.setString(3, order.details);
                },
                JdbcExecutionOptions.builder()
                        .withBatchSize(1000)
                        .withBatchIntervalMs(200) // Quick "write-behind"
interval
                        .build(),
                new JdbcConnectionOptions.JdbcConnectionOptionsBuilder()
                        .withUrl("jdbc:mysql://localhost:3306/order_db")
                        .withDriverName("com.mysql.cj.jdbc.Driver")
                        .withUsername("app_user")
                        .withPassword("secret")
                        .build()
        ));

        // 5. Execute the program [2, 12]
        // Triggers the lazy execution of the entire JobGraph [13, 14]
        env.execute("Order Processing and Write-Behind Sync");
    }
}
```

Key Elements of This Structure:

- **Lazy Execution:** The `addSource`, `keyBy`, and `addSink` calls do not process data immediately; they merely build the logical **JobGraph**[121]. The heavy lifting only starts when `env.execute()` is called[122].
- **Fluent Chaining:** The transformations are applied in a single, readable pipeline, similar to the "route" concept in Apache Camel[123][124].
- **Idempotency for Fast Write-Behind:** By using an **UPSERT** SQL query in the JDBC sink, the application ensures that even if Flink replays a record after a failure, the RDBMS remains eventually consistent with

Flink's internal state without manual conflict resolution[125][126][127].

- **Reliability:** Explicitly enabling **checkpointing** in Step 1 ensures that Flink's internal `ValueState` (the "Order" objects) can be recovered even if cluster nodes fail[128][129].

How would you write it if there are multiple workflows similar to the order processing one and the application was a spring boot app?

Integrating Apache Flink with **Spring Boot** allows you to leverage Spring's dependency injection and configuration management to handle **multiple workflows** (or "routes") in a modular, maintainable way[130][131]. Instead of one giant `main()` method, you can define each workflow as a separate **Spring Bean** and use a central runner to assemble them[132].

Here is how you would structure an application with multiple workflows (e.g., Order Processing and Payment Processing):

1. Define a Workflow Interface

Create a common interface for your streaming pipelines. This makes it easy to inject all defined workflows into the Flink engine automatically.

```
public interface FlinkWorkflow {
    void build(StreamExecutionEnvironment env);
}
```

2. Implement Modular Workflows as Spring Components

Each workflow is its own class, where you can `@Autowired` Spring services or use `@Value` to inject configuration from `application.properties` (like Kafka topics or database URLs)[133][134].

```
@Component
public class OrderWorkflow implements FlinkWorkflow {

    @Value("${kafka.topic.orders}")
    private String orderTopic;

    @Override
    public void build(StreamExecutionEnvironment env) {
        env.addSource(new FlinkKafkaConsumer<>(orderTopic, new OrderSchema(),
...))
           .keyBy(Order::getOrderId)
           .process(new OrderProcessor())
           .addSink(new JdbcSink(...));
    }
}

@Component
public class PaymentWorkflow implements FlinkWorkflow {

    @Value("${kafka.topic.payments}")
    private String paymentTopic;
```

```
    @Override
    public void build(StreamExecutionEnvironment env) {
        env.addSource(new FlinkKafkaConsumer<>(paymentTopic, new PaymentSchema(),
...))
            .keyBy(Payment::getTransactionId)
            .process(new PaymentProcessor())
            .addSink(new SomeOtherSink(...));
    }
}
```

3. Create a Central Flink Configuration

Use Spring's `@Configuration` to provide the `StreamExecutionEnvironment` as a bean. This is where you set cluster-wide settings like **checkpointing**[132][135].

```
@Configuration
public class FlinkConfig {

    @Bean
    public StreamExecutionEnvironment flinkEnvironment() {
        StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();
        env.enableCheckpointing(5000); // Standard fault tolerance setup
        return env;
    }
}
```

4. Assemble and Run via `CommandLineRunner`

Use a `CommandLineRunner` to gather all `FlinkWorkflow` beans and trigger the execution[133]. This separates the Spring application's startup from the Flink job's lifecycle[133].

```
@Component
public class FlinkJobLauncher implements CommandLineRunner {

    @Autowired
    private StreamExecutionEnvironment env;

    @Autowired
    private List<FlinkWorkflow> workflows; // Spring injects all implementations

    @Override
    public void run(String... args) throws Exception {
        // Step 1: Add all modular workflows to the same execution environment
        for (FlinkWorkflow workflow : workflows) {
            workflow.build(env);
        }
```

```
        // Step 2: Trigger the lazy execution of the combined JobGraph [7]
        env.execute("Multi-Workflow Streaming App");
    }
}
```

Key Advantages of This Spring Setup:

- **Modularization:** Adding a new workflow (e.g., a "ShippingWorkflow") simply requires creating a new `@Component` class; you don't need to modify the main runner logic.
- **Property Injection:** Use Spring's `application.properties` or `YAML` files to manage environment-specific settings (like dev vs. production Kafka brokers)[135].
- **Resource Management:** Spring handles the "plumbing" (connecting to databases, managing REST endpoints), while Flink focuses strictly on the complex event-driven logic[131].
- **Testing:** You can use Spring Boot's testing framework to load a partial context and run integration tests on specific workflows using Flink's test harness[135].

**Important Deployment Note:** When packaging this for a Flink cluster, set the Flink dependencies to **provided** scope in your Gradle/Maven file to prevent version conflicts with the cluster's runtime[136].

Who calls the run method inside FlinkJobLauncher and what does the main class look like now with Flink + Spring Boot?

In a Spring Boot integration, the `run` method inside the `FlinkJobLauncher` is **automatically called by the Spring Boot framework** immediately after the application context has finished starting up[137]. This is because the `FlinkJobLauncher` implements the **CommandLineRunner** interface, which Spring Boot specifically looks for to trigger tasks at startup.

The Spring Boot Main Class

With this architecture, your `main` class becomes a standard, minimal Spring Boot entry point. It no longer contains the Flink dataflow logic; instead, it simply bootstraps the Spring context, which then discovers your Flink components (like the `FlinkJobLauncher` and various workflows) as beans[137] [138].

The main class typically looks like this:

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class FlinkStreamingApplication {

    public static void main(String[] args) {
        // This call bootstraps the Spring context.
        // Once initialized, Spring Boot will automatically find the
        // FlinkJobLauncher bean and call its run() method.
        SpringApplication.run(FlinkStreamingApplication.class, args);
    }
}
```

Why this structure is used:

- **Separation of Concerns:** Flink handles the complex stream processing logic (the "JobGraph"), while Spring Boot handles the "plumbing"—configuration, database connections, and REST APIs[139] [140].
- **Control:** This approach separates the Spring application's startup from the Flink job's lifecycle, giving you more control over resource management[137].
- **Developer Experience:** You can use standard Spring features like **@Autowired** for services and **@Value** for configuration properties (such as Kafka brokers or database URLs) directly within your Flink workflow classes[141].

---

## Sources

1. Stream Processing with Apache Flink.pdf
2. Stream Processing with Apache Flink.pdf
3. Stream Processing with Apache Flink.pdf
4. Stream Processing with Apache Flink.pdf
5. Stream Processing with Apache Flink.pdf
6. Stream Processing with Apache Flink.pdf
7. Stream Processing with Apache Flink.pdf
8. Stream Processing with Apache Flink.pdf
9. Stream Processing with Apache Flink.pdf
10. Stream Processing with Apache Flink.pdf
11. Stream Processing with Apache Flink.pdf
12. Stream Processing with Apache Flink.pdf
13. Stream Processing with Apache Flink.pdf
14. Stream Processing with Apache Flink.pdf
15. Stream Processing with Apache Flink.pdf
16. Stream Processing with Apache Flink.pdf
17. Stream Processing with Apache Flink.pdf
18. Stream Processing with Apache Flink.pdf
19. Stream Processing with Apache Flink.pdf
20. Stream Processing with Apache Flink.pdf

> **Types | 97**
>
> type information to represent data types and generate specific serializers, deserializ- ers, and comparators for every data type.
> Flink also features a type extraction system that analyzes the input and return types of functions to automatically obtain type information and hence serializers and deserial- izers. However, in certain situations, such as lambda functions or generic types, it is necessary to explicitly provide type information to make an application work or improve its performance.

21. Stream Processing with Apache Flink.pdf

> In this section, we discuss the types supported by Flink, how to create type informa- tion for a data type, and how to help Flink's type system with hints if it cannot auto- matically infer the return type of a function.
> **Supported Data Types** Flink supports all common data types that are available in Java and Scala. The most widely used types can be grouped into the following categories:
> • Primitives
> • Java and Scala tuples
> • Scala case classes
> • POJOs, including classes generated by Apache Avro
> • Some special types

22. Stream Processing with Apache Flink.pdf

> **Hello, Flink!**
> Let's start with a simple example to get a first impression of what it is like to write streaming applications with the DataStream API. We will use this example to show- case the basic structure of a Flink program and introduce some important features of the DataStream API. Our example application ingests a stream of temperature meas- urements from multiple sensors.
> First, let's have a look at the data type we will be using to represent sensor readings:
> **case class SensorReading( id: String, timestamp: Long, temperature: Double)**

23. Stream Processing with Apache Flink.pdf

• It is a public class.
• It has a public constructor without any arguments—a default constructor.
• All fields are public or accessible through getters and setters. The getter and setter functions must follow the default naming scheme, which is Y getX() and setX(Y x) for a field x of type Y.
• All fields have types that are supported by Flink. For example, the following Java class will be identified as a POJO by Flink: Avro-generated classes are automatically identified by Flink and handled as POJOs.

24. Stream Processing with Apache Flink.pdf

*Example 8-9. POJO class with Cassandra Object Mapper annotations*
In addition to the configuration options in Figures 8-7 and 8-8, a Cassandra sink builder provides a few more methods to configure the sink connector:
• setClusterBuilder(ClusterBuilder): The ClusterBuilder builds a Cassandra Cluster that manages the connection to Cassandra. Among other options, it can configure the hostnames and ports of one or more contact points; define load balancing, retry, and reconnection policies; and provide access credentials.
• setHost(String, [Int]): This method is a shortcut for a simple Cluster Builder configured with the hostname and port of a single contact point. If no port is configured, Cassandra's default port 9042 is used.
• setQuery(String): This specifies the CQL INSERT query to write tuples, case classes, or rows to Cassandra. A query must not be configured to emit POJOs.
• setMapperOptions(MapperOptions): This provides options for Cassandra's Object Mapper, such as configurations for consistency, time-to-live (TTL), and null field handling. The options are ignored if the sink emits tuples, case classes, or rows.
• enableWriteAheadLog([CheckpointCommitter]): This enables the WAL to pro- vide exactly-once output guarantees in the case of nondeterministic application logic. CheckpointCommitter is used to store information about completed check-

25. Stream Processing with Apache Flink.pdf

**Bootstrap a Flink Maven Project**
Importing the examples-scala repository into your IDE to experiment with Flink is a good first step. However, you should also know how to create a new Flink project from scratch.
Flink provides Maven archetypes to generate Maven projects for Java or Scala Flink applications. Open a terminal and run the following command to create a Flink Maven Quickstart Scala project as a starting point for your Flink application:

```
mvn archetype:generate                               \
    -DarchetypeGroupId=org.apache.flink              \
    -DarchetypeArtifactId=flink-quickstart-scala     \
    -DarchetypeVersion=1.7.1                         \
    -DgroupId=org.apache.flink.quickstart            \
    -DartifactId=flink-scala-project                 \
    -Dversion=0.1                                    \
```

26. Stream Processing with Apache Flink.pdf

```
src/ └── main      ├── resources      |    └── log4j.properties      └── scala          └── org
└── apache                  └── flink                       └── quickstart
├── BatchJob.scala                          └── StreamingJob.scala
```

*The project contains two skeleton files, BatchJob.scala and StreamingJob.scala, as a* starting point for your own programs. You can also delete them if you do not need them.
You can import the project in your IDE following the steps we described in the previ- ous section or you can execute the following command to build a JAR file:

27. Stream Processing with Apache Flink.pdf

```
import org.apache.flink.streaming.api.scala._
```

**Types | 101**
**Explicitly Providing Type Information** In most cases, Flink can automatically infer types and generate the correct TypeInfor mation. Flink's type extractor leverages reflection and analyzes function signatures and subclass information to derive the correct output type of a user-defined function. Sometimes, though, the necessary information cannot be extracted (e.g., because of Java erasing generic type information). Moreover, in

some cases Flink might not `choose the TypeInformation that generates the most efficient serializers and deserial- izers. Hence, you might need to explicitly provide TypeInformation objects to Flink` for some of the data types used in your application.

28. Stream Processing with Apache Flink.pdf

In order to grasp the problem of modifying the data type of a state, we have to under- stand how state data is represented within a savepoint. A savepoint consists mainly of serialized state data. The serializers that convert the state JVM objects into bytes are generated and configured by Flink's type system. This conversion is based on the data `type of the state. For example, if you have a ValueState[String], Flink's type system generates a StringSerializer to convert String objects into bytes. The serializer is` also used to convert the raw bytes back into JVM objects. Depending on whether the `state backend stores the data serialized (like the RocksDBStateBackend) or as objects on the heap (like the FSStateBackend), this happens when the state is read by a func-` tion or when an application is restarted from a savepoint.

29. Stream Processing with Apache Flink.pdf

• It is a public class.
• It has a public constructor without any arguments—a default constructor.
• All fields are public or accessible through getters and setters. The getter and `setter functions must follow the default naming scheme, which is Y getX() and setX(Y x) for a field x of type Y.`
• All fields have types that are supported by Flink. For example, the following Java class will be identified as a POJO by Flink: Avro-generated classes are automatically identified by Flink and handled as POJOs.

30. Stream Processing with Apache Flink.pdf

In this section, we discuss the types supported by Flink, how to create type informa- tion for a data type, and how to help Flink's type system with hints if it cannot auto- matically infer the return type of a function.
**Supported Data Types** Flink supports all common data types that are available in Java and Scala. The most widely used types can be grouped into the following categories:
• Primitives
• Java and Scala tuples
• Scala case classes
• POJOs, including classes generated by Apache Avro
• Some special types

31. Stream Processing with Apache Flink.pdf



type information to represent data types and generate specific serializers, deserializ- ers, and comparators for every data type.
Flink also features a type extraction system that analyzes the input and return types of functions to automatically obtain type information and hence serializers and deserial- izers. However, in certain situations, such as lambda functions or generic types, it is necessary to explicitly provide type information to make an application work or improve its performance.

32. Stream Processing with Apache Flink.pdf

Types that are not specially handled are treated as generic types and serialized using the Kryo serialization framework.
**Only Use Kryo as a Fallback Solution**
Note that you should avoid using Kryo if possible. Since Kryo is a general-purpose serializer it is usually not very efficient. Flink pro- vides configuration options to improve the efficiency by preregis- tering classes to Kryo. Moreover, Kryo does not provide a good migration path to evolve data types.
Let's look at each type category.
*Primitives* `All Java and Scala primitive types, such as Int (or Integer for Java), String, and Double, are supported. Here is an example that processes a stream of Long values` and increments each element:

33. Stream Processing with Apache Flink.pdf

There are two ways to provide TypeInformation. First, you can extend a function class to explicitly provide the TypeInformation of its return type by implementing the ResultTypeQueryable interface. The following example shows a MapFunction that provides its return type:
In the Java DataStream API, you can also use the returns() method to explicitly specify the return type of an operator when defining the dataflow as shown in the fol- lowing:
DataStream<Tuple2<String, Integer>> tuples = ... DataStream<Person> persons = tuples **.map(t -> new Person(t.f0, t.f1))** *// provide TypeInformation for the map lambda function's return type*
.returns(Types.POJO(Person.class));

34. Stream Processing with Apache Flink.pdf

*Example 8-9. POJO class with Cassandra Object Mapper annotations*
In addition to the configuration options in Figures 8-7 and 8-8, a Cassandra sink builder provides a few more methods to configure the sink connector:
• setClusterBuilder(ClusterBuilder): The ClusterBuilder builds a Cassandra Cluster that manages the connection to Cassandra. Among other options, it can configure the hostnames and ports of one or more contact points; define load balancing, retry, and reconnection policies; and provide access credentials.
• setHost(String, [Int]): This method is a shortcut for a simple Cluster Builder configured with the hostname and port of a single contact point. If no port is configured, Cassandra's default port 9042 is used.
• setQuery(String): This specifies the CQL INSERT query to write tuples, case classes, or rows to Cassandra. A query must not be configured to emit POJOs.
• setMapperOptions(MapperOptions): This provides options for Cassandra's Object Mapper, such as configurations for consistency, time-to-live (TTL), and null field handling. The options are ignored if the sink emits tuples, case classes, or rows.
• enableWriteAheadLog([CheckpointCommitter]): This enables the WAL to pro- vide exactly-once output guarantees in the case of nondeterministic application logic. CheckpointCommitter is used to store information about completed check-

35. Stream Processing with Apache Flink.pdf

**val numbers: DataStream[Long] = env.fromElements(1L, 2L, 3L, 4L) numbers.map( n => n + 1)**
*Java and Scala tuples* Tuples are composite data types that consist of a fixed number of typed fields.

The Scala DataStream API uses regular Scala tuples. The following example fil- ters a DataStream of tuples with two fields:
Flink provides efficient implementations of Java tuples. Flink's Java tuples can have up to 25 fields, with each length is implemented as a separate class—Tuple1, Tuple2, up to Tuple25. The tuple classes are strongly typed.

36. Stream Processing with Apache Flink.pdf

**Hello, Flink!**
Let's start with a simple example to get a first impression of what it is like to write streaming applications with the DataStream API. We will use this example to show- case the basic structure of a Flink program and introduce some important features of the DataStream API. Our example application ingests a stream of temperature meas- urements from multiple sensors.
First, let's have a look at the data type we will be using to represent sensor readings:
**case class SensorReading( id: String, timestamp: Long, temperature: Double)**

37. Stream Processing with Apache Flink.pdf

We can rewrite the filtering example in the Java DataStream API as follows:
Tuple fields can be accessed by the name of their public fields—f0, f1, f2, etc., as shown earlier—or by position using the getField(int pos) method, where indexes start at 0:

`Tuple2<String, Integer> personTuple = Tuple2.of("Alex", "42");` *Integer age = personTuple.getField(1); // age = 42*
In contrast to their Scala counterparts, Flink's Java tuples are mutable, so the val- ues of fields can be reassigned. Functions can reuse Java tuples in order to reduce the pressure on the garbage collector. The following example shows how to update a field of a Java tuple:

38. Stream Processing with Apache Flink.pdf

*personTuple.f1 = 42; // set the 2nd field to 42 personTuple.setField(43, 1); // set the 2nd field to 43*
*Scala case classes* Flink supports Scala case classes. Case class fields are accessed by name. In the `following, we define a case class Person with two fields: name and age. As for the tuples, we filter the DataStream by age:`

*// filter for persons with age > 18* **persons.filter(p => p.age > 18)**
*POJOs* Flink analyzes each type that does not fall into any category and checks to see if it can be identified and handled as a POJO type. Flink accepts a class as a POJO if it satisfies the following conditions:

39. Stream Processing with Apache Flink.pdf

• It is a public class.
• It has a public constructor without any arguments—a default constructor.
• All fields are public or accessible through getters and setters. The getter and `setter functions must follow the default naming scheme, which is Y getX() and setX(Y x) for a field x of type Y.`
• All fields have types that are supported by Flink. For example, the following Java class will be identified as a POJO by Flink: Avro-generated classes are automatically identified by Flink and handled as POJOs.

40. Stream Processing with Apache Flink.pdf

*Arrays, Lists, Maps, Enums, and other special types* `Flink supports several special-purpose types, such as primitive and object Array types; Java's ArrayList, HashMap, and Enum types; and Hadoop Writable types. Moreover, it provides type information for Scala's Either, Option, and Try types, and Flink's Java version of the Either type.`
**Creating Type Information for Data Types** `The central class in Flink's type system is TypeInformation. It provides the system` with the necessary information it needs to generate serializers and comparators. For

41. Stream Processing with Apache Flink.pdf

Types that are not specially handled are treated as generic types and serialized using the Kryo serialization framework.
**Only Use Kryo as a Fallback Solution**
Note that you should avoid using Kryo if possible. Since Kryo is a general-purpose serializer it is usually not very efficient. Flink pro- vides configuration options to improve the efficiency by preregis- tering classes to Kryo. Moreover, Kryo does not provide a good migration path to evolve data types.
Let's look at each type category.
*Primitives* `All Java and Scala primitive types, such as Int (or Integer for Java), String, and Double, are supported. Here is an example that processes a stream of Long values` and increments each element:

42. Stream Processing with Apache Flink.pdf

The application logic to read from and write to state is often straightforward. How- ever, efficient and reliable management of state is more challenging. This includes handling of very large states, possibly exceeding memory, and ensuring that no state is lost in case of failures. All issues related to state consistency, failure handling, and efficient storage and access are taken care of by Flink so that developers can focus on the logic of their applications.
In Flink, state is always associated with a specific operator. In order to make Flink's runtime aware of the state of an operator, the operator needs to register its state. *There are two types of state, operator state and keyed state, that are accessible from* different scopes and discussed in the following sections.

43. Stream Processing with Apache Flink.pdf

**State Management | 53**

**Operator State** Operator state is scoped to an operator task. This means that all records processed by the same parallel task have access to the same state. Operator state cannot be accessed by another task of the same or a different operator. Figure 3-11 shows how tasks access operator state.
*Figure 3-11. Tasks with operator state*
Flink offers three primitives for operator state:
*List state* Represents state as a list of entries.
*Union list state* Represents state as a list of entries as well. But it differs from regular list state in how it is restored in the case of a failure or when an application is started from a savepoint. We discuss this difference later in this chapter.

44. Stream Processing with Apache Flink.pdf

*Broadcast state* Designed for the special case where the state of each task of an operator is identi- cal. This property can be leveraged during checkpoints and when rescaling an operator. Both aspects are discussed in later sections of this chapter.
**Keyed State** Keyed state is maintained and accessed with respect to a key defined in the records of an operator's input stream. Flink maintains one state instance per key value and parti- tions all records with the same key to the operator task that maintains the state for this key. When a task processes a record, it automatically scopes the state access to the

45. Stream Processing with Apache Flink.pdf

**156 | Chapter 7: Stateful Operators and Applications**

2 The serialization format of state is an important aspect when updating an application and is discussed later in this chapter.
`specify a TypeSerializer to control how state is written into a state backend, check-` point, and savepoint.2
`Typically, the state handle object is created in the open() method of RichFunction. open() is called before any processing methods, such as flatMap() in the case of a FlatMapFunction, are called. The state handle object (lastTempState in` Example 7-2) is a regular member variable of the function class.

46. Stream Processing with Apache Flink.pdf

The state handle object only provides access to the state, which is stored an maintained in the state backend. The handle does not hold the state itself.
`When a function registers a StateDescriptor, Flink checks if the state backend has` data for the function and a state with the given name and type. This might happen if the stateful function is restarted to recover from a failure or when an application is started from a savepoint. In both cases, Flink links the newly registered state handle object to the existing state. If the state backend does not contain state for the given descriptor, the state that is linked to the handle is initialized as empty.

47. Stream Processing with Apache Flink.pdf

`Example 7-2 shows the implementation of a FlatMapFunction with a keyed Value State that checks whether the measured temperature changed more than a config-` ured threshold.
**Implementing Stateful Functions | 155**
*Example 7-2. Implementing a FlatMapFunction with a keyed ValueState*
`To create a state object, we have to register a StateDescriptor with Flink's runtime via the RuntimeContext, which is exposed by RichFunction (see "Implementing Functions" on page 105 for a discussion of the RichFunction interface). The StateDe scriptor is specific to the state primitive and includes the name of the state and the data types of the state. The descriptors for ReducingState and AggregatingState also need a ReduceFunction or AggregateFunction object to aggregate the added` values. The state name is scoped to the operator so that a function can have more than one state object by registering multiple state descriptors. The data types handled `by the state are specified as Class or TypeInformation objects (see "Types" on page` 97 for a discussion of Flink's type handling). The data type must be specified because Flink needs to create a suitable serializer. Alternatively, it is also possible to explicitly

48. Stream Processing with Apache Flink.pdf

• ValueState[T] holds a single value of type T. The value can be read using Value State.value() and updated with ValueState.update(value: T).
• ListState[T] holds a list of elements of type T. New elements can be appended to the list by calling ListState.add(value: T) or ListState.addAll(values: java.util.List[T]). The state elements can be accessed by calling List State.get(), which returns an Iterable[T] over all state elements. It is not pos- sible to remove individual elements from ListState, but the list can be updated by calling ListState.update(values: java.util.List[T]). A call to this method will replace existing values with the given list of values.
• MapState[K, V] holds a map of keys and values. The state primitive offers many of the methods of a regular Java Map such as get(key: K), put(key: K, value:

49. Stream Processing with Apache Flink.pdf


V), contains(key: K), remove(key: K), and iterators over the contained entries, keys, and values.
• ReducingState[T] offers the same methods as ListState[T] (except for addAll() and update()), but instead of appending values to a list, Reducing State.add(value: T) immediately aggregates value using a ReduceFunction. The iterator returned by get() returns an Iterable with a single entry, which is the reduced value.
• AggregatingState[I, O] behaves similar to ReducingState. However, it uses the more general AggregateFunction to aggregate values. Aggregating

50. Stream Processing with Apache Flink.pdf

StateStore and a KeyedStateStore object. The state stores are responsible for regis- tering function state with Flink's runtime and returning the state objects, such as ValueState, ListState, or BroadcastState. Each state is registered with a name that must be unique for the function. When a function registers state, the state store tries to initialize the state by checking if the state backend holds state for the function reg- istered under the given name. If the task is restarted due to a failure or from a save- point, the state will be initialized from the saved data. If the application is not started from a checkpoint or savepoint, the state will be initially empty.

51. Stream Processing with Apache Flink.pdf

A function can work with operator list state by implementing the ListCheckpointed interface. The ListCheckpointed interface does not work with state handles like Val ueState or ListState, which are registered at the state backend. Instead, functions implement operator state as regular member variables and interact with the state backend via callback functions of the ListCheckpointed interface. The interface pro- vides two methods:
The snapshotState() method is invoked when Flink triggers a checkpoint of the stateful function. The method has two parameters, checkpointId, which is a unique, monotonically increasing identifier for checkpoints, and timestamp, which is the wall-clock time when the master initiated the checkpoint. The method has to return the operator state as a list of serializable state objects.

52. Stream Processing with Apache Flink.pdf

In this section, we discuss the types supported by Flink, how to create type informa- tion for a data type, and how to help Flink's type system with hints if it cannot auto- matically infer the return type of a function.
**Supported Data Types** Flink supports all common data types that are available in Java and Scala. The most widely used types can be grouped into the following categories:
• Primitives
• Java and Scala tuples
• Scala case classes
• POJOs, including classes generated by Apache Avro
• Some special types

53. Stream Processing with Apache Flink.pdf

**Types | 97**

type information to represent data types and generate specific serializers, deserializ- ers, and comparators for every data type.
Flink also features a type extraction system that analyzes the input and return types of functions to automatically obtain type information and hence serializers and deserial- izers. However, in certain situations, such as lambda functions or generic types, it is necessary to explicitly provide type information to make an application work or improve its performance.

54. Stream Processing with Apache Flink.pdf

*Example 7-7. Implementing a KeyedBroadcastProcessFunction*
**162 | Chapter 7: Stateful Operators and Applications**

`BroadcastProcessFunction and KeyedBroadcastProcessFunction differ from a regular CoProcessFunction because the element processing methods are not sym- metric. The methods, processElement() and processBroadcastElement(), are called with different context objects. Both context objects offer a method getBroad castState(MapStateDescriptor) that provides access to a broadcast state handle. However, the broadcast state handle that is returned in the processElement()` method provides read-only access to the broadcast state. This is a safety mechanism to ensure the broadcast state holds the same information in all parallel instances. In addition, both context objects also provide access to the event-time timestamp, the current watermark, the current processing time, and the side outputs, similar to the context objects of other process functions.

55. Stream Processing with Apache Flink.pdf

**164 | Chapter 7: Stateful Operators and Applications**
`Example 7-8 shows how the CheckpointedFunction interface is used to create a func-` tion with keyed and operator state that counts per key and operator instance how many sensor readings exceed a specified threshold.
*Example 7-8. A function implementing the CheckpointedFunction interface*
**Implementing Stateful Functions | 165**

**Receiving Notifications About Completed Checkpoints** Frequent synchronization is a major reason for performance limitations in dis- tributed systems. Flink's design aims to reduce synchronization points. Checkpoints are implemented based on barriers that flow with the data and therefore avoid global synchronization across all operators of an application.

56. Stream Processing with Apache Flink.pdf

**State Management | 55**
that is called a state backend. A state backend is responsible for two things: local state management and checkpointing state to a remote location.
For local state management, a state backend stores all keyed states and ensures that all accesses are correctly scoped to the current key. Flink provides state backends that manage keyed state as objects stored in in-memory data structures on the JVM heap. Another state backend serializes state objects and puts them into RocksDB, which writes them to local hard disks. While the first option gives very fast state access, it is limited by the size of the memory. Accessing state stored by the RocksDB state back- end is slower but its state may grow very large.

57. Stream Processing with Apache Flink.pdf

State checkpointing is important because Flink is a distributed system and state is only locally maintained. A TaskManager process (and with it, all tasks running on it) may fail at any point in time. Hence, its storage must be considered volatile. A state backend takes care of checkpointing the state of a task to a remote and persistent storage. The remote storage for checkpointing could be a distributed filesystem or a database system. State backends differ in how state is checkpointed. For instance, the RocksDB state backend supports incremental checkpoints, which can significantly reduce the checkpointing overhead for very large state sizes.

58. Stream Processing with Apache Flink.pdf

**58 | Chapter 3: The Architecture of Apache Flink**

**Consistent Checkpoints** Flink's recovery mechanism is based on consistent checkpoints of application state. A consistent checkpoint of a stateful streaming application is a copy of the state of each of its tasks at a point when all tasks have processed exactly the same input. This can be explained by looking at the steps of a naive algorithm that takes a consistent checkpoint of an application. The steps of this naive algorithm would be:
1. Pause the ingestion of all input streams. 2. Wait for all in-flight data to be completely processed, meaning all tasks have pro-

59. Stream Processing with Apache Flink.pdf

Typical use cases for event-driven applications include:
• Real-time recommendations (e.g., for recommending products while customers browse a retailer's website)
• Pattern detection or complex event processing (e.g., for fraud detection in credit card transactions)
• Anomaly detection (e.g., to detect attempts to intrude a computer network)
Event-driven applications are an evolution of microservices. They communicate via event logs instead of REST calls and hold application data as local state instead of writing it to and reading it from an external datastore, such as a relational database or key-value store. Figure 1-5 shows a service architecture composed of event-driven streaming applications.

60. Stream Processing with Apache Flink.pdf



*Figure 1-5. An event-driven application architecture*
The applications in Figure 1-5 are connected by event logs. One application emits its output to an event log and another application consumes the events the other appli- cation emitted. The event log decouples senders and receivers and provides asynchro- nous, nonblocking event transfer. Each application can be stateful and can locally manage its own state without accessing external datastores. Applications can also be individually operated and scaled.

61. Stream Processing with Apache Flink.pdf

*Broadcast state* Designed for the special case where the state of each task of an operator is identi- cal. This property can be leveraged during checkpoints and when rescaling an operator. Both aspects are discussed in later sections of this chapter.
**Keyed State** Keyed state is maintained and accessed with respect to a key defined in the records of an operator's input stream. Flink maintains one state instance per key value and parti- tions all records with the same key to the operator task that maintains the state for this key. When a task processes a record, it automatically scopes the state access to the

62. Stream Processing with Apache Flink.pdf

In data streaming, latency is measured in units of time, such as milliseconds. Depend- *ing on the application, you might care about average latency, maximum latency, or percentile latency. For example, an average latency value of 10 ms means that events* are processed within 10 ms on average. Alternately, a 95th-percentile latency value of 10 ms means that 95% of events are processed within 10 ms. Average values hide the true distribution of processing delays and might make it hard to detect problems. If the barista runs out of milk right before preparing your cappuccino, you will have to

63. Stream Processing with Apache Flink.pdf

• It is a public class.
• It has a public constructor without any arguments—a default constructor.
• All fields are public or accessible through getters and setters. The getter and
`setter functions must follow the default naming scheme, which is Y getX() and setX(Y x) for a field x of type Y.`
• All fields have types that are supported by Flink. For example, the following Java class will be identified as a POJO by Flink:
Avro-generated classes are automatically identified by Flink and handled as POJOs.

64. Stream Processing with Apache Flink.pdf

1. At the source: Timestamps and watermarks can be assigned and generated by a `SourceFunction when a stream is ingested into an application. A source function` emits a stream of records. Records can be emitted together with an associated timestamp, and watermarks can be emitted at any point in time as special records. If a source function (temporarily) does not emit anymore watermarks, it can declare itself idle. Flink will exclude stream partitions produced by idle source functions from the watermark computation of subsequent operators. The idle mechanism of sources can be used to address the problem of not advancing watermarks as discussed earlier. Source functions are discussed in more detail in "Implementing a Custom Source Function" on page 202.

65. Stream Processing with Apache Flink.pdf

**Provided Connectors | 187**

The dependency for the universal Flink Kafka connector is added to a Maven project as shown in the following:
**<dependency> <groupId>org.apache.flink</groupId> <artifactId>flink-connector-kafka_2.12</artifactId> <version>1.7.1</version> </dependency>**
The Flink Kafka connector ingests event streams in parallel. Each parallel source task can read from one or more partitions. A task tracks for each partition its current reading offset and includes it into its checkpoint data. When recovering from a failure, the offsets are restored and the source instance continues reading from the checkpointed offset. The Flink Kafka connector does not rely on Kafka's own offsettracking mechanism, which is based on so-called consumer groups. Figure 8-1 shows the assignment of partitions to source instances.

66. Stream Processing with Apache Flink.pdf

*Figure 1-8. Screenshot of Apache Flink's web dashboard showing the overview*
**A Quick Look at Flink | 13**

5. Download the JAR file that includes examples in this book: `$ wget https://streaming-with-flink.github.io/\ examples/download/examples-scala.jar`
You can also build the JAR file yourself by following the steps in the repository's README file.
6. Run the example on your local cluster by specifying the application's entry class and JAR file:
```
$ ./bin/flink run \   -c io.github.streamingwithflink.chapter1.AverageSensorReadings \
examples-scala.jar Starting execution of program Job has been submitted with JobID
cfde9dbe315ce162444c475a08cf93d9
```

67. Stream Processing with Apache Flink.pdf

**keyBy**
`The keyBy transformation converts a DataStream into a KeyedStream by specifying a` key. Based on the key, the events of the stream are assigned to partitions, so that all events with the same key are processed by the same task of the subsequent operator. Events with different keys can be processed by the same task, but the keyed state of a task's function is always accessed in the scope of the current event's key.
Considering the color of the input event as the key, Figure 5-4 assigns black events to one partition and all other events to another partition.

68. Stream Processing with Apache Flink.pdf

`Currently, Flink provides eight different process functions: ProcessFunction, Keyed ProcessFunction, CoProcessFunction, ProcessJoinFunction, BroadcastProcess Function, KeyedBroadcastProcessFunction, ProcessWindowFunction, and ProcessAllWindowFunction. As indicated by name, these functions are applicable in` different contexts. However, they have a very similar feature set. We will continue dis- `cussing these common features by looking in detail at the KeyedProcessFunction.`
`The KeyedProcessFunction is a very versatile function and can be applied to a Keyed Stream. The function is called for each record of the stream and returns zero, one, or more records. All process functions implement the RichFunction interface and hence offer open(), close(), and getRuntimeContext() methods. Additionally, a KeyedPro cessFunction[KEY, IN, OUT] provides the following two methods:`

69. Stream Processing with Apache Flink.pdf

> • `ValueState[T]` holds a single value of type T. The value can be read using Value State.value() and updated with ValueState.update(value: T).
> • `ListState[T]` holds a list of elements of type T. New elements can be appended to the list by calling ListState.add(value: T) or ListState.addAll(values: java.util.List[T]). The state elements can be accessed by calling List State.get(), which returns an Iterable[T] over all state elements. It is not pos- sible to remove individual elements from ListState, but the list can be updated by calling ListState.update(values: java.util.List[T]). A call to this method will replace existing values with the given list of values.
> • `MapState[K, V]` holds a map of keys and values. The state primitive offers many of the methods of a regular Java Map such as get(key: K), put(key: K, value:

70. Stream Processing with Apache Flink.pdf

> **vi | Table of Contents**
> Apache Cassandra Sink Connector 199 Implementing a Custom Source Function 202
> Resettable Source Functions 203 Source Functions, Timestamps, and Watermarks 204
> Implementing a Custom Sink Function 206 Idempotent Sink Connectors 207 Transactional Sink Connectors 209

71. Stream Processing with Apache Flink.pdf

> Flink provides connectors for Apache Kafka, Kinesis, RabbitMQ, Apache Nifi, vari- ous filesystems, Cassandra, ElasticSearch, and JDBC. In addition, the Apache Bahir project provides additional Flink connectors for ActiveMQ, Akka, Flume, Netty, and Redis.
> In order to use a provided connector in your application, you need to add its depend- ency to the build file of your project. We explained how to add connector dependen- cies in "Including External and Flink Dependencies" on page 107.
> In the following section, we discuss the connectors for Apache Kafka, file-based sour- ces and sinks, and Apache Cassandra. These are the most widely used connectors and they also represent important types of source and sink systems. You can find more information about the other connectors in the documentation for Apache Flink or Apache Bahir.

72. Stream Processing with Apache Flink.pdf

> 2. The external system supports updates per key, such as a relational database sys- tem or a key-value store.
> `Example 8-13 illustrates how to implement and use an idempotent SinkFunction that` writes to a JDBC database, in this case an embedded Apache Derby database.
> **Implementing a Custom Sink Function | 207**
> *Example 8-13. An idempotent SinkFunction that writes to a JDBC database*
> **208 | Chapter 8: Reading from and Writing to External Systems**
> row with the given key exists. The Cassandra sink connector follows the same approach when the WAL is not enabled.

73. Stream Processing with Apache Flink.pdf

> **184 | Chapter 8: Reading from and Writing to External Systems**
> 2 We discuss the consistency guarantees of a WAL sink in more detail in "GenericWriteAheadSink".
> contains the key-value pair. On the other hand, an append operation is not an idem- potent operation, because appending an element multiple times results in multiple appends. Idempotent write operations are interesting for streaming applications because they can be performed multiple times without changing the results. Hence, they can to some extent mitigate the effect of replayed results as caused by Flink's checkpointing mechanism.

74. Stream Processing with Apache Flink.pdf

> Flink provides a sink connector to write data streams to Cassandra. Cassandra's data model is based on primary keys, and all writes to Cassandra happen with upsert semantics. In combination with exactly-once checkpointing, resettable sources, and deterministic application logic, upsert writes yield eventually exactly-once output consistency. The output is only eventually consistent, because results are reset to a previous version during recovery, meaning consumers might read older results than read previously. Also, the versions of values for multiple keys might be out of sync.

75. Stream Processing with Apache Flink.pdf

It should be noted an application that relies on idempotent sinks to achieve exactlyonce results must guarantee that it overrides previously written results while it replays. For example, an application with a sink that upserts into a key-value store must ensure that it deterministically computes the keys that are used to upsert. More- over, applications that read from the sink system might observe unexpected results during the time when an application recovers. When the replay starts, previously emitted results might be overridden by earlier results. Hence, an application that con- sumes the output of the recovering application might witness a jump back in time, e.g., read a smaller count than before. Also, the overall result of the streaming appli- cation will be in an inconsistent state while the replay is in progress because some results will be overridden while others are not. Once the replay completes and the application is past the point at which it previously failed, the result will be consistent again.

76. Stream Processing with Apache Flink.pdf

**Idempotent Sink Connectors** `For many applications, the SinkFunction interface is sufficient to implement an` idempotent sink connector. This is possible if the following two properties hold:
1. The result data has a deterministic (composite) key, on which idempotent updates can be performed. For an application that computes the average temper- ature per sensor and minute, a deterministic key could be the ID of the sensor and the timestamp for each minute. Deterministic keys are important to ensure all writes are correctly overwritten in case of a recovery.

77. Stream Processing with Apache Flink.pdf

Instead of waiting to be periodically triggered, a streaming analytics application con- tinuously ingests streams of events and updates its result by incorporating the latest events with low latency. This is similar to the maintenance techniques database sys-


tems use to update materialized views. Typically, streaming applications store their result in an external data store that supports efficient updates, such as a database or key-value store. The live updated results of a streaming analytics application can be used to power dashboard applications as shown in Figure 1-6.

78. Stream Processing with Apache Flink.pdf

**Transactional Writes** The second approach to achieve end-to-end exactly-once consistency is based on transactional writes. The idea here is to only write those results to an external sink system that have been computed before the last successful checkpoint. This behavior guarantees end-to-end exactly-once because in case of a failure, the application is reset to the last checkpoint and no results have been emitted to the sink system after that checkpoint. By only writing data once a checkpoint is completed, the transac- tional approach does not suffer from the replay inconsistency of the idempotent writes. However, it adds latency because results only become visible when a check- point completes.

79. Stream Processing with Apache Flink.pdf

Providing exactly-once guarantees requires at-least-once guarantees, and thus a data replay mechanism is again necessary. Additionally, the stream processor needs to ensure internal state consistency. That is, after recovery, it should know whether an event update has already been reflected on the state or not. Transactional updates are one way to achieve this result, but they can incur substantial performance overhead. Instead, Flink uses a lightweight snapshotting mechanism to achieve exactly-once result guarantees. We discuss Flink's fault-tolerance algorithm in "Checkpoints, Save- points, and State Recovery" on page 58.

80. Stream Processing with Apache Flink.pdf



*Example 7-9. Enabling checkpointing for an application*

The checkpointing interval is an important parameter that affects the overhead of the checkpointing mechanism during regular processing and the time it takes to recover from a failure. A shorter checkpointing interval causes higher overhead during regu- lar processing but can enable faster recovery because less data needs to be reproc- essed.
Flink provides more tuning knobs to configure the checkpointing behavior, such as the choice of consistency guarantees (exactly-once or at-least-once), the number of concurrent checkpoints, and a timeout to cancel long-running checkpoints, as well as several state backend–specific options. We discuss these options in more detail in “Tuning Checkpointing and Recovery” on page 263.

81. Stream Processing with Apache Flink.pdf

Apache Flink stores the application state locally in memory or in an embedded data- base. Since Flink is a distributed system, the local state needs to be protected against failures to avoid data loss in case of application or machine failure. Flink guarantees this by periodically writing a consistent checkpoint of the application state to a remote and durable storage. State, state consistency, and Flink’s checkpointing mech- anism will be discussed in more detail in the following chapters, but, for now, Figure 1-4 shows a stateful streaming Flink application.

82. Stream Processing with Apache Flink.pdf

**System Architecture**
Flink is a distributed system for stateful parallel data stream processing. A Flink setup consists of multiple processes that typically run distributed across multiple machines. Common challenges that distributed systems need to address are allocation and man- agement of compute resources in a cluster, process coordination, durable and highly available data storage, and failure recovery.
Flink does not implement all this functionality by itself. Instead, it focuses on its core function—distributed data stream processing—and leverages existing cluster infra- structure and services. Flink is well integrated with cluster resource managers, such as Apache Mesos, YARN, and Kubernetes, but can also be configured to run as a standalone cluster. Flink does not provide durable, distributed storage. Instead, it takes advantage of distributed filesystems like HDFS or object stores such as S3. For leader election in highly available setups, Flink depends on Apache ZooKeeper.

83. Stream Processing with Apache Flink.pdf

This checkpointing and recovery mechanism can provide exactly-once consistency for application state, given that all operators checkpoint and restore all of their states

and that all input streams are reset to the position up to which they were consumed when the checkpoint was taken. Whether a data source can reset its input stream depends on its implementation and the external system or interface from which the stream is consumed. For instance, event logs like Apache Kafka can provide records from a previous offset of the stream. In contrast, a stream consumed from a socket cannot be reset because sockets discard data once it has been consumed. Conse- quently, an application can only be operated under exactly-once state consistency if all input streams are consumed by resettable data sources.

84. Stream Processing with Apache Flink.pdf

In order to provide exactly-once state consistency for an application,1 each source connector of the application needs to be able to set its read positions to a previously checkpointed position. When taking a checkpoint, a source operator persists its read- ing positions and restores these positions during recovery. Examples for source con- nectors that support the checkpointing of reading positions are file-based sources that store the reading offsets in the byte stream of the file or a Kafka source that stores the reading offsets in the topic partitions it consumes. If an application ingests data from a source connector that is not able to store and reset a reading position, it might suffer from data loss in the case of a failure and only provide at-most-once guarantees.

85. Stream Processing with Apache Flink.pdf

*Figure 8-1. Read offsets of Kafka topic partitions*
A Kafka source connector is created as shown in Example 8-1.
*Example 8-1. Creating a Flink Kafka source*
The constructor takes three arguments. The first argument defines the topics to read from. This can be a single topic, a list of topics, or a regular expression that matches all topics to read from. When reading from multiple topics, the Kafka connector treats all partitions of all topics the same and multiplexes their events into a single stream.

86. Stream Processing with Apache Flink.pdf

• registerEventTimeTimer(timestamp: Long): Unit registers an event-time timer for the current key. The timer will fire when the watermark is updated to a timestamp that is equal to or larger than the timer's timestamp.
• deleteProcessingTimeTimer(timestamp: Long): Unit deletes a processing-time timer that was previously registered for the current key. If no such timer exists, the method has no effect.
• deleteEventTimeTimer(timestamp: Long): Unit deletes an event-time timer that was previously registered for the current key. If no such timer exists, the method has no effect.

87. Stream Processing with Apache Flink.pdf

This means that you should take application requirements and the properties of its input data, such as key domain, into account when designing and implementing stateful operators. If your application requires keyed state for a moving key domain, it should ensure the state of keys is cleared when it is not needed anymore. This can be done by registering timers for a point of time in the future.4 Similar to state, timers are registered in the context of the currently active key. When the timer fires, a call- back method is called and the context of timer's key is loaded. Hence, the callback method has full access to the key's state and can also clear it. The functions that offer support to register timers are the Trigger interface for windows and the process function. Both were covered in Chapter 6.

88. Stream Processing with Apache Flink.pdf

Since our KeyedProcessFunction always updates the registered timer by deleting the current timer and registering a new one, only a single timer is registered per key. Once that timer fires, the onTimer() method is called. The method clears all state associated with the key, the last temperature and the last timer state.

**Evolving Stateful Applications**

It is often necessary to fix a bug or to evolve the business logic of a long-running stateful streaming application. Consequently, a running application needs to be replaced by an updated version usually without losing the state of the application.

89. Stream Processing with Apache Flink.pdf

*Figure 5-3. A flatMap operation that outputs white squares, duplicates black squares, and drops gray squares*

The flatMap transformation applies a function on each incoming event. The corre- sponding FlatMapFunction defines the flatMap() method, which may return zero, one, or more events as results by passing them to the Collector object:

```
// T: the type of input elements // O: the type of output elements FlatMapFunction[T, O]
> flatMap(T, Collector[O]): Unit
```

This example shows a flatMap transformation commonly found in data processing tutorials. The function is applied on a stream of sentences, splits each sentence by the space character, and emits each resulting word as an individual record:

90. Stream Processing with Apache Flink.pdf

**116 | Chapter 6: Time-Based and Window Operators**

**TimerService and Timers** The TimerService of the Context and OnTimerContext objects offers the following methods:
• currentProcessingTime(): Long returns the current processing time.
• currentWatermark(): Long returns the timestamp of the current watermark.
• registerProcessingTimeTimer(timestamp: Long): Unit registers a processing time timer for the current key. The timer will fire when the processing time of the executing machine reaches the provided timestamp.

91. Stream Processing with Apache Flink.pdf

When a program is submitted for execution, all function objects are serialized using Java serialization and shipped to all parallel tasks of their corresponding operators. Therefore, all configuration values are preserved after the object is deserialized.

**Implementing Functions | 105**

**Functions Must Be Java Serializable**
Flink serializes all function objects with Java serialization to ship them to the worker processes. Everything contained in a user func- `tion must be Serializable.` If your function requires a nonserializable object instance, you can either implement it as a rich function and initialize the nonserializ- `able field in the open() method or override the Java serialization` and deserialization methods.

92. Stream Processing with Apache Flink.pdf

**Lambda Functions** Most DataStream API methods accept lambda functions. Lambda functions are avail- able for Scala and Java and offer a simple and concise way to implement application logic when no advanced operations such as accessing state and configuration are required. The following example show a lambda function that filters tweets contain- ing the word "flink":
**val tweets: DataStream[String] = ...** *// a filter lambda function that checks if tweets contains the word "flink"* **val flinkTweets = tweets.filter(_.contains("flink"))**

93. Stream Processing with Apache Flink.pdf



2 The serialization format of state is an important aspect when updating an application and is discussed later in this chapter.
`specify a TypeSerializer to control how state is written into a state backend, check-` point, and savepoint.2
`Typically, the state handle object is created in the open() method of RichFunction. open() is called before any processing methods, such as flatMap() in the case of a FlatMapFunction, are called. The state handle object (lastTempState in` Example 7-2) is a regular member variable of the function class.

94. Stream Processing with Apache Flink.pdf

The state handle object only provides access to the state, which is stored an maintained in the state backend. The handle does not hold the state itself.
`When a function registers a StateDescriptor, Flink checks if the state backend has` data for the function and a state with the given name and type. This might happen if the stateful function is restarted to recover from a failure or when an application is started from a savepoint. In both cases, Flink links the newly registered state handle object to the existing state. If the state backend does not contain state for the given descriptor, the state that is linked to the handle is initialized as empty.

95. Stream Processing with Apache Flink.pdf

`Keyed state can only be used by functions that are applied on a KeyedStream. A Keyed Stream is constructed by calling the DataStream.keyBy() method that defines a key on a stream. A KeyedStream is partitioned on the specified key and remembers the key definition. An operator that is applied on a KeyedStream is applied in the context` of its key definition.
Flink provides multiple primitives for keyed state. A state primitive defines the struc- ture of the state for an individual key. The choice of the right state primitive depends on how the function interacts with the state. The choice also affects the performance of a function because each state backend provides its own implementations for these primitives. The following state primitives are supported by Flink:

96. Stream Processing with Apache Flink.pdf

`• ValueState[T] holds a single value of type T. The value can be read using Value State.value() and updated with ValueState.update(value: T).`
`• ListState[T] holds a list of elements of type T. New elements can be appended to the list by calling ListState.add(value: T) or ListState.addAll(values: java.util.List[T]). The state elements can be accessed by calling List State.get(), which returns an Iterable[T] over all state elements. It is not pos- sible to remove individual elements from ListState, but the list can be updated by calling ListState.update(values: java.util.List[T]). A call to this` method will replace existing values with the given list of values.

```
• MapState[K, V] holds a map of keys and values. The state primitive offers many of the
methods of a regular Java Map such as get(key: K), put(key: K, value:
```

97. Stream Processing with Apache Flink.pdf

> The application logic to read from and write to state is often straightforward. How- ever, efficient and reliable management of state is more challenging. This includes handling of very large states, possibly exceeding memory, and ensuring that no state is lost in case of failures. All issues related to state consistency, failure handling, and efficient storage and access are taken care of by Flink so that developers can focus on the logic of their applications.
> In Flink, state is always associated with a specific operator. In order to make Flink's runtime aware of the state of an operator, the operator needs to register its state. *There are two types of state, operator state and keyed state, that are accessible from* different scopes and discussed in the following sections.

98. Stream Processing with Apache Flink.pdf

> **State Management | 53**
>
> **Operator State** Operator state is scoped to an operator task. This means that all records processed by the same parallel task have access to the same state. Operator state cannot be accessed by another task of the same or a different operator. Figure 3-11 shows how tasks access operator state.
> *Figure 3-11. Tasks with operator state*
> Flink offers three primitives for operator state:
> *List state* Represents state as a list of entries.
> *Union list state* Represents state as a list of entries as well. But it differs from regular list state in how it is restored in the case of a failure or when an application is started from a savepoint. We discuss this difference later in this chapter.

99. Stream Processing with Apache Flink.pdf

> **54 | Chapter 3: The Architecture of Apache Flink**
>
> key of the current record. Consequently, all records with the same key access the same state. Figure 3-12 shows how tasks interact with keyed state.
> *Figure 3-12. Tasks with keyed state*
> You can think of keyed state as a key-value map that is partitioned (or sharded) on the key across all parallel tasks of an operator. Flink provides different primitives for keyed state that determine the type of the value stored for each key in this distributed key-value map. We will briefly discuss the most common keyed state primitives.

100. Stream Processing with Apache Flink.pdf

> *Value state* Stores a single value of arbitrary type per key. Complex data structures can also be stored as value state.
> *List state* Stores a list of values per key. The list entries can be of arbitrary type.
> *Map state* Stores a key-value map per key. The key and value of the map can be of arbitrary type.
> State primitives expose the structure of the state to Flink and enable more efficient state accesses. They are discussed further in "Declaring Keyed State at RuntimeCon- text" on page 154.
> **State Backends** A task of a stateful operator typically reads and updates its state for each incoming record. Because efficient state access is crucial to processing records with low latency, each parallel task locally maintains its state to ensure fast state accesses. How exactly the state is stored, accessed, and maintained is determined by a pluggable component

101. Stream Processing with Apache Flink.pdf

> **Queryable State**
> Many stream processing applications need to share their results with other applica- tions. A common pattern is to write results into a database or key-value store and have other applications retrieve the result from that datastore. Such an architecture implies that a separate system needs to be set up and maintained, which can be a major effort, especially if this needs to be a distributed system as well.
> Apache Flink features queryable state to address use cases that usually would require an external datastore to share data. In Flink, any keyed state can be exposed to exter- nal applications as queryable state and act as a read-only key-value store. The stateful streaming application processes events as usual and stores and updates its intermedi- ate or final results in a queryable state. External applications can request the state for a key while the streaming application is running.

102. Stream Processing with Apache Flink.pdf

> Note that only key point queries are supported. It is not possible to request key ranges or even run more complex queries. Queryable state does not address all use cases that require an external datastore. For example, the queryable state is only accessible while the application is running. It is not accessible while the application is restarted due to an error, for rescaling the application, or to migrate it to another cluster. However, it makes many applications much easier to realize, such as real-time dashboards or other monitoring applica- tions.

103. Stream Processing with Apache Flink.pdf

> **Introduction to Dataflow Programming**
> Before we delve into the fundamentals of stream processing, let's look at the back- *ground on dataflow programming and the terminology we will use throughout this* book.
> **Dataflow Graphs** As the name suggests, a dataflow program describes how data flows between opera- tions. Dataflow programs are commonly represented as directed graphs, where nodes are called operators and represent computations and edges represent data dependen- cies. Operators are the basic functional units of a dataflow application. They consume data from inputs, perform a computation on them, and produce data to outputs for further processing. Operators without input ports are called data sources and opera- tors without output ports are called data sinks. A dataflow graph must have at least one data source and one data sink. Figure 2-1 shows a dataflow program that extracts and counts hashtags from an input stream of tweets.

104. Stream Processing with Apache Flink.pdf

> The application logic to read from and write to state is often straightforward. How- ever, efficient and reliable management of state is more challenging. This includes handling of very large states, possibly exceeding memory, and ensuring that no state is lost in case of failures. All issues related to state consistency, failure handling, and efficient storage and access are taken care of by Flink so that developers can focus on the logic of their applications.
> In Flink, state is always associated with a specific operator. In order to make Flink's runtime aware of the state of an operator, the operator needs to register its state. *There are two types of state, operator state and keyed state, that are accessible from* different scopes and discussed in the following sections.

105. Stream Processing with Apache Flink.pdf

> **State Management | 53**
>
> **Operator State** Operator state is scoped to an operator task. This means that all records processed by the same parallel task have access to the same state. Operator state cannot be accessed by another task of the same or a different operator. Figure 3-11 shows how tasks access operator state.
> *Figure 3-11. Tasks with operator state*
> Flink offers three primitives for operator state:
> *List state* Represents state as a list of entries.
> *Union list state* Represents state as a list of entries as well. But it differs from regular list state in how it is restored in the case of a failure or when an application is started from a savepoint. We discuss this difference later in this chapter.

106. Stream Processing with Apache Flink.pdf

> Note that only key point queries are supported. It is not possible to request key ranges or even run more complex queries. Queryable state does not address all use cases that require an external datastore. For example, the queryable state is only accessible while the application is running. It is not accessible while the application is restarted due to an error, for rescaling the application, or to migrate it to another cluster. However, it makes many applications much easier to realize, such as real-time dashboards or other monitoring applica- tions.

107. Stream Processing with Apache Flink.pdf

> `Example 7-2 shows the implementation of a FlatMapFunction with a keyed Value State that checks whether the measured temperature changed more than a config-` ured threshold.
> **Implementing Stateful Functions | 155**
> *Example 7-2. Implementing a FlatMapFunction with a keyed ValueState*
> `To create a state object, we have to register a StateDescriptor with Flink's runtime via the RuntimeContext, which is exposed by RichFunction (see "Implementing Functions" on page 105`

for a discussion of the RichFunction interface). The StateDe scriptor is specific to the state primitive and includes the name of the state and the data types of the state. The descriptors for ReducingState and AggregatingState also need a ReduceFunction or AggregateFunction object to aggregate the added values. The state name is scoped to the operator so that a function can have more than one state object by registering multiple state descriptors. The data types handled by the state are specified as Class or TypeInformation objects (see “Types” on page 97 for a discussion of Flink’s type handling). The data type must be specified because Flink needs to create a suitable serializer. Alternatively, it is also possible to explicitly

108. Stream Processing with Apache Flink.pdf

**Queryable State**
Many stream processing applications need to share their results with other applica- tions. A common pattern is to write results into a database or key-value store and have other applications retrieve the result from that datastore. Such an architecture implies that a separate system needs to be set up and maintained, which can be a major effort, especially if this needs to be a distributed system as well.
Apache Flink features queryable state to address use cases that usually would require an external datastore to share data. In Flink, any keyed state can be exposed to exter- nal applications as queryable state and act as a read-only key-value store. The stateful streaming application processes events as usual and stores and updates its intermedi- ate or final results in a queryable state. External applications can request the state for a key while the streaming application is running.

109. Stream Processing with Apache Flink.pdf

**val client: QueryableStateClient = new QueryableStateClient(tmHostname, proxyPort)**
Once you obtain a state client, you can query the state of an application by calling the getKvState() method. The method takes several parameters, such as the JobID of the running application, the state identifier, the key for which the state should be fetched, the TypeInformation for the key, and the StateDescriptor of the queried state. The JobID can be obtained via the REST API, the Web UI, or the log files. The getKvState() method returns a CompletableFuture[S] where S is the type of the state (e.g., ValueState[_] or MapState[_, _]). Hence, the client can send out multi- ple asynchronous queries and wait for their results. Example 7-16 shows a simple console dashboard that queries the queryable state of the application shown in the previous section.

110. Stream Processing with Apache Flink.pdf

The program in Example 5-1 converts the temperatures from Fahrenheit to Celsius and computes the average temperature every 5 seconds for each sensor.
**The DataStream API (v1.7) | 79**
*Example 5-1. Compute the average temperature every 5 seconds for a stream of sensors*
You have probably already noticed that Flink programs are defined and submitted for execution in regular Scala or Java methods. Most commonly, this is done in a static main method. In our example, we define the AverageSensorReadings object and include most of the application logic inside main().

111. Stream Processing with Apache Flink.pdf

To structure a typical Flink streaming application:
1. Set up the execution environment.
**80 | Chapter 5: The DataStream API (v1.7)**
2. Read one or more streams from data sources. 3. Apply streaming transformations to implement the application logic. 4. Optionally output the result to one or more data sinks. 5. Execute the program.
We now look at these parts in detail.
**Set Up the Execution Environment** *The first thing a Flink application needs to do is set up its execution environment. The* execution environment determines whether the program is running on a local machine or on a cluster. In the DataStream API, the execution environment of an application is represented by the StreamExecutionEnvironment. In our example, we retrieve the execution environment by calling the static getExecutionEnvironment() method. This method returns a local or remote environment, depending on the con- text in which the method is invoked. If the method is invoked from a submission cli- ent with a connection to a remote cluster, a remote execution environment is returned. Otherwise, it returns a local environment.

112. Stream Processing with Apache Flink.pdf

marks(new SensorTimeAssigner) method assigns the timestamps and watermarks that are required for event time. The implementation details of SensorTimeAssigner do not concern us right now.
**Apply Transformations** Once we have a DataStream, we can apply a transformation on it. There are different types of transformations. Some transformations can produce a new DataStream, possibly of a different type, while other transformations do not modify the records of the DataStream but reorganize it by partitioning or grouping. The logic of an application is defined by chaining transformations.

113. Stream Processing with Apache Flink.pdf

In our example, we first apply a map() transformation that converts the temperature of each sensor reading to Celsius. Then, we use the keyBy() transformation to parti- tion the sensor readings by their sensor ID. Next, we define a timeWindow() transfor- mation, which groups the sensor readings of each sensor ID partition into tumbling windows of 5 seconds:
**val avgTemp: DataStream[SensorReading] = sensorData .map( r => { val celsius = (r.temperature - 32) * (5.0 / 9.0) SensorReading(r.id, r.timestamp, celsius) } ) .keyBy(_.id) .timeWindow(Time.seconds(5)) .apply(new TemperatureAverager)**

114. Stream Processing with Apache Flink.pdf

**83**
Stream API program essentially boils down to combining such transformations to create a dataflow graph that implements the application logic.
Most stream transformations are based on user-defined functions. The functions encapsulate the user application logic and define how the elements of the input stream are transformed into the elements of the output stream. Functions, such as MapFunction in the following, are defined as classes that implement a transformation-specific function interface:
**class MyMapFunction extends MapFunction[Int, Int] { override def map(value: Int): Int = value + 1 }**

115. Stream Processing with Apache Flink.pdf

**Data ingestion and data egress** Data ingestion and data egress operations allow the stream processor to communicate *with external systems. Data ingestion is the operation of fetching raw data from exter-* nal sources and converting it into a format suitable for processing. Operators that *implement data ingestion logic are called data sources. A data source can ingest data from a TCP socket, a file, a Kafka topic, or a sensor data interface. Data egress is the* operation of producing output in a form suitable for consumption by external sys- *tems. Operators that perform data egress are called data sinks and examples include* files, databases, message queues, and monitoring interfaces.

116. Stream Processing with Apache Flink.pdf

2. Periodic assigner: The DataStream API provides a user-defined function called AssignerWithPeriodicWatermarks that extracts a timestamp from each record and is periodically queried for the current watermark. The extracted timestamps are assigned to the respective record and the queried watermarks are ingested into the stream. This function will be discussed in “Assigning Timestamps and Generating Watermarks” on page 111.
3. Punctuated assigner: AssignerWithPunctuatedWatermarks is another user- defined function that extracts a timestamp from each record. It can be used to generate watermarks that are encoded in special input records. In contrast to the AssignerWithPeriodicWatermarks function, this function can—but does not need to—extract a watermark from each record. We discuss this function in detail in “Assigning Timestamps and Generating Watermarks” as well.

117. Stream Processing with Apache Flink.pdf

**82 | Chapter 5: The DataStream API (v1.7)**

implement your own streaming sinks. There are also applications that do not emit results but keep them internally to serve them via Flink’s queryable state feature.

In our example, the result is a DataStream[SensorReading] record. Every record contains an average temperature of a sensor over a period of 5 seconds. The result stream is written to the standard output by calling print():

```
avgTemp.print()
```

Note that the choice of a streaming sink affects the end-to-end con- sistency of an application, whether the result of the application is provided with at-least once or exactly-once semantics. The end-to-end consistency of the application depends on the integration of the chosen stream sinks with Flink's checkpointing algorithm. We will discuss this topic in more detail in "Application Consistency Guarantees" on page 184.

118. Stream Processing with Apache Flink.pdf

The constructed plan is translated into a JobGraph and submitted to a JobManager for execution. Depending on the type of execution environment, a JobManager is started as a local thread (local execution environment) or the JobGraph is sent to a remote JobManager. If the JobManager runs remotely, the JobGraph must be shipped together with a JAR file that contains all classes and required dependencies of the application.

**Transformations**

In this section we give an overview of the basic transformations of the DataStream API. Time-related operators such as window operators and other specialized trans- formations are described in later chapters. A stream transformation is applied on one or more streams and converts them into one or more output streams. Writing a Data-

119. Stream Processing with Apache Flink.pdf

The program in Example 5-1 converts the temperatures from Fahrenheit to Celsius and computes the average temperature every 5 seconds for each sensor.

**The DataStream API (v1.7) | 79**

*Example 5-1. Compute the average temperature every 5 seconds for a stream of sensors*

You have probably already noticed that Flink programs are defined and submitted for execution in regular Scala or Java methods. Most commonly, this is done in a static main method. In our example, we define the AverageSensorReadings object and include most of the application logic inside main().

120. Stream Processing with Apache Flink.pdf

To structure a typical Flink streaming application:

1. Set up the execution environment.

**80 | Chapter 5: The DataStream API (v1.7)**

2. Read one or more streams from data sources. 3. Apply streaming transformations to implement the application logic. 4. Optionally output the result to one or more data sinks. 5. Execute the program.

We now look at these parts in detail.

**Set Up the Execution Environment** *The first thing a Flink application needs to do is set up its execution environment. The* execution environment determines whether the program is running on a local machine or on a cluster. In the DataStream API, the execution environment of an application is represented by the StreamExecutionEnvironment. In our example, we retrieve the execution environment by calling the static getExecutionEnvironment() method. This method returns a local or remote environment, depending on the con- text in which the method is invoked. If the method is invoked from a submission cli- ent with a connection to a remote cluster, a remote execution environment is returned. Otherwise, it returns a local environment.

121. Stream Processing with Apache Flink.pdf

The constructed plan is translated into a JobGraph and submitted to a JobManager for execution. Depending on the type of execution environment, a JobManager is started as a local thread (local execution environment) or the JobGraph is sent to a remote JobManager. If the JobManager runs remotely, the JobGraph must be shipped together with a JAR file that contains all classes and required dependencies of the application.

**Transformations**

In this section we give an overview of the basic transformations of the DataStream API. Time-related operators such as window operators and other specialized trans- formations are described in later chapters. A stream transformation is applied on one or more streams and converts them into one or more output streams. Writing a Data-

122. Stream Processing with Apache Flink.pdf

**Execute** When the application has been completely defined, it can be executed by call- `ing StreamExecutionEnvironment.execute(). This is the last call in our example: env.execute("Compute average sensor temperature")`
Flink programs are executed lazily. That is, the API calls that create stream sources and transformations do not immediately trigger any data processing. Instead, the API calls construct an execution plan in the execution environment, which consists of the stream sources created from the environment and all transformations that were tran- `sitively applied to these sources. Only when execute() is called does the system trig-` ger the execution of the program.

123. Stream Processing with Apache Flink.pdf

`In our example, we first apply a map() transformation that converts the temperature of each sensor reading to Celsius. Then, we use the keyBy() transformation to parti- tion the sensor readings by their sensor ID. Next, we define a timeWindow() transfor-` mation, which groups the sensor readings of each sensor ID partition into tumbling windows of 5 seconds:
**val avgTemp: DataStream[SensorReading] = sensorData .map( r => { val celsius = (r.temperature - 32) * (5.0 / 9.0) SensorReading(r.id, r.timestamp, celsius) } ) .keyBy(_.id) .timeWindow(Time.seconds(5)) .apply(new TemperatureAverager)**

124. Stream Processing with Apache Flink.pdf

**83**
Stream API program essentially boils down to combining such transformations to create a dataflow graph that implements the application logic.
Most stream transformations are based on user-defined functions. The functions encapsulate the user application logic and define how the elements of the input stream are transformed into the elements of the output stream. Functions, such as `MapFunction in the following, are defined as classes that implement a` transformation-specific function interface:
**class MyMapFunction extends MapFunction[Int, Int] { override def map(value: Int): Int = value + 1** }

125. Stream Processing with Apache Flink.pdf

2. The external system supports updates per key, such as a relational database sys- tem or a key-value store.
`Example 8-13 illustrates how to implement and use an idempotent SinkFunction that` writes to a JDBC database, in this case an embedded Apache Derby database.
**Implementing a Custom Sink Function | 207**
*Example 8-13. An idempotent SinkFunction that writes to a JDBC database*
**208 | Chapter 8: Reading from and Writing to External Systems**
row with the given key exists. The Cassandra sink connector follows the same approach when the WAL is not enabled.

126. Stream Processing with Apache Flink.pdf

It should be noted an application that relies on idempotent sinks to achieve exactlyonce results must guarantee that it overrides previously written results while it replays. For example, an application with a sink that upserts into a key-value store must ensure that it deterministically computes the keys that are used to upsert. More- over, applications that read from the sink system might observe unexpected results during the time when an application recovers. When the replay starts, previously emitted results might be overridden by earlier results. Hence, an application that con- sumes the output of the recovering application might witness a jump back in time, e.g., read a smaller count than before. Also, the overall result of the streaming appli- cation will be in an inconsistent state while the replay is in progress because some results will be overridden while others are not. Once the replay completes and the application is past the point at which it previously failed, the result will be consistent again.

127. Stream Processing with Apache Flink.pdf

Flink provides a sink connector to write data streams to Cassandra. Cassandra's data model is based on primary keys, and all writes to Cassandra happen with upsert semantics. In combination with exactly-once checkpointing, resettable sources, and deterministic application logic, upsert writes yield eventually exactly-once output consistency. The output is only eventually consistent, because results are reset to a previous version during recovery, meaning consumers might read older results than read previously. Also, the versions of values for multiple keys might be out of sync.

128. Stream Processing with Apache Flink.pdf

> **166 | Chapter 7: Stateful Operators and Applications**
>
> *Example 7-9. Enabling checkpointing for an application*
> The checkpointing interval is an important parameter that affects the overhead of the checkpointing mechanism during regular processing and the time it takes to recover from a failure. A shorter checkpointing interval causes higher overhead during regu- lar processing but can enable faster recovery because less data needs to be reproc- essed.
> Flink provides more tuning knobs to configure the checkpointing behavior, such as the choice of consistency guarantees (exactly-once or at-least-once), the number of concurrent checkpoints, and a timeout to cancel long-running checkpoints, as well as several state backend–specific options. We discuss these options in more detail in “Tuning Checkpointing and Recovery” on page 263.

129. Stream Processing with Apache Flink.pdf

> In order to provide exactly-once state consistency for an application,1 each source connector of the application needs to be able to set its read positions to a previously checkpointed position. When taking a checkpoint, a source operator persists its read- ing positions and restores these positions during recovery. Examples for source con- nectors that support the checkpointing of reading positions are file-based sources that store the reading offsets in the byte stream of the file or a Kafka source that stores the reading offsets in the topic partitions it consumes. If an application ingests data from a source connector that is not able to store and reset a reading position, it might suffer from data loss in the case of a failure and only provide at-most-once guarantees.

130. How to Integrate Apache Flink with Spring Boot for Real-Time Stream Processing | Elite Dev

> I've been thinking about stream processing lately. Not as an abstract concept, but as a practical need. In my work, I keep seeing the same pattern: data arrives continuously, decisions need to be made in real-time, and the old batch-processing mindset just doesn't fit. This led me to explore how we can build these systems without starting from scratch every time. How can we use tools we already know to solve new problems?
> Java (Programming Language)
> Discover more
> java
> Development Tools
> Java
> That's where the idea of bringing Apache Flink into a Spring Boot application came from. Flink handles the complex stream processing logic, while Spring Boot provides the familiar structure and utilities for building the application around it. It's about combining specialized power with general-purpose comfort.

131. How to Integrate Apache Flink with Spring Boot for Real-Time Stream Processing | Elite Dev

> Computer Science
> Let's start with the basics. What is Flink doing in this setup? At its core, Flink processes sequences of events. Imagine a never-ending flow of information—sensor readings, financial trades, website clicks. Flink can read this flow, perform calculations, detect patterns, and output results, all as the data is moving. Spring Boot, on the other hand, is the home for your application. It manages configuration, connects to databases, exposes REST APIs, and handles all the standard plumbing. By integrating them, you let each tool do what it's best at.

132. How to Integrate Apache Flink with Spring Boot for Real-Time Stream Processing | Elite Dev

> Data Management
> Now, where does your processing logic live? In Flink, you build a job. This job is a graph of operations—sources, transformations, and sinks. In a Spring Boot application, you can define this job as a Spring Bean. This is a clean way to let the Spring context manage its creation and configuration.
> This code creates a simple stream that converts text to uppercase. But here's a question: how do you actually run this? You don't want the Flink job to start immediately when the Spring application starts. You need control.

133. How to Integrate Apache Flink with Spring Boot for Real-Time Stream Processing | Elite Dev

> Java (Programming Language)
> A common approach is to create a command-line runner or a dedicated REST endpoint that triggers the job execution. This separates the application's startup from the Flink job's lifecycle, which is important for managing resources and deployment.

This runner will execute the Flink job when the application starts. But what if your data source isn't a simple list? In reality, your data is probably coming from a message queue like Apache Kafka. This is where the integration shows its strength. You can use Spring's configuration properties to define your Kafka connection and inject them into your Flink job setup.

134. How to Integrate Apache Flink with Spring Boot for Real-Time Stream Processing | Elite Dev

Open Source
Does this mean every Spring Boot app needs Flink? No. This integration is for a specific type of problem: when you have continuous, high-volume data that requires immediate, stateful analysis. It's for building real-time dashboards, fraud detection systems, or live data pipelines.
The real benefit is developer experience. You can focus on writing your business logic—the transformations, aggregations, and patterns you need to find—while relying on Spring for the rest of the application concerns. You use `@Autowired` for your services, JPA repositories for database access, and standard Spring properties for configuration, all while Flink's engine handles the streaming data.

135. How to Integrate Apache Flink with Spring Boot for Real-Time Stream Processing | Elite Dev

Programming
Think about state. One of Flink's key features is its ability to remember information across events—like a running total or the last seen value. When you run Flink within a Spring Boot application, you need to consider where this state is stored. For development, the local filesystem might be fine. For production, you'd configure a distributed storage backend, like Hadoop HDFS or S3. Spring's property files can help manage these environment-specific settings.
Testing is another critical piece. How do you verify your stream logic works? Flink provides a test harness that lets you feed test data into your stream and check the outputs. You can combine this with Spring Boot's testing framework to load your application context and run integrated tests on your processing pipeline.

136. How to Integrate Apache Flink with Spring Boot for Real-Time Stream Processing | Elite Dev

Distributed & Cloud Computing
Discover more
DATA
Company News
Network Monitoring & Management
So, how do you begin? The first step is to add Flink to your Spring Boot project. You manage this through your build tool, like Maven or Gradle. This brings the Flink libraries into your application's classpath.
Software
It's crucial to set the scope to `provided` . This tells your build tool that Flink will be available in the runtime environment, preventing version conflicts when you deploy your job to a Flink cluster.

137. How to Integrate Apache Flink with Spring Boot for Real-Time Stream Processing | Elite Dev

Java (Programming Language)
A common approach is to create a command-line runner or a dedicated REST endpoint that triggers the job execution. This separates the application's startup from the Flink job's lifecycle, which is important for managing resources and deployment.
This runner will execute the Flink job when the application starts. But what if your data source isn't a simple list? In reality, your data is probably coming from a message queue like Apache Kafka. This is where the integration shows its strength. You can use Spring's configuration properties to define your Kafka connection and inject them into your Flink job setup.

138. How to Integrate Apache Flink with Spring Boot for Real-Time Stream Processing | Elite Dev

Data Management
Now, where does your processing logic live? In Flink, you build a job. This job is a graph of operations—sources, transformations, and sinks. In a Spring Boot application, you can define this job as a Spring Bean. This is a clean way to let the Spring context manage its creation and configuration.
This code creates a simple stream that converts text to uppercase. But here's a question: how do you actually run this? You don't want the Flink job to start immediately when the Spring application starts. You need control.

139. How to Integrate Apache Flink with Spring Boot for Real-Time Stream Processing | Elite Dev

I've been thinking about stream processing lately. Not as an abstract concept, but as a practical need. In my work, I keep seeing the same pattern: data arrives continuously, decisions need to be made in real-time, and the old batch-processing

mindset just doesn't fit. This led me to explore how we can build these systems without starting from scratch every time. How can we use tools we already know to solve new problems?
Java (Programming Language)
Discover more
java
Development Tools
Java
That's where the idea of bringing Apache Flink into a Spring Boot application came from. Flink handles the complex stream processing logic, while Spring Boot provides the familiar structure and utilities for building the application around it. It's about combining specialized power with general-purpose comfort.

140. How to Integrate Apache Flink with Spring Boot for Real-Time Stream Processing | Elite Dev

Computer Science
Let's start with the basics. What is Flink doing in this setup? At its core, Flink processes sequences of events. Imagine a never-ending flow of information—sensor readings, financial trades, website clicks. Flink can read this flow, perform calculations, detect patterns, and output results, all as the data is moving. Spring Boot, on the other hand, is the home for your application. It manages configuration, connects to databases, exposes REST APIs, and handles all the standard plumbing. By integrating them, you let each tool do what it's best at.

141. How to Integrate Apache Flink with Spring Boot for Real-Time Stream Processing | Elite Dev

Open Source
Does this mean every Spring Boot app needs Flink? No. This integration is for a specific type of problem: when you have continuous, high-volume data that requires immediate, stateful analysis. It's for building real-time dashboards, fraud detection systems, or live data pipelines.
The real benefit is developer experience. You can focus on writing your business logic—the transformations, aggregations, and patterns you need to find—while relying on Spring for the rest of the application concerns. You use `@Autowired` for your services, JPA repositories for database access, and standard Spring properties for configuration, all while Flink's engine handles the streaming data.